Lab #8

KestrelSAT Capstone

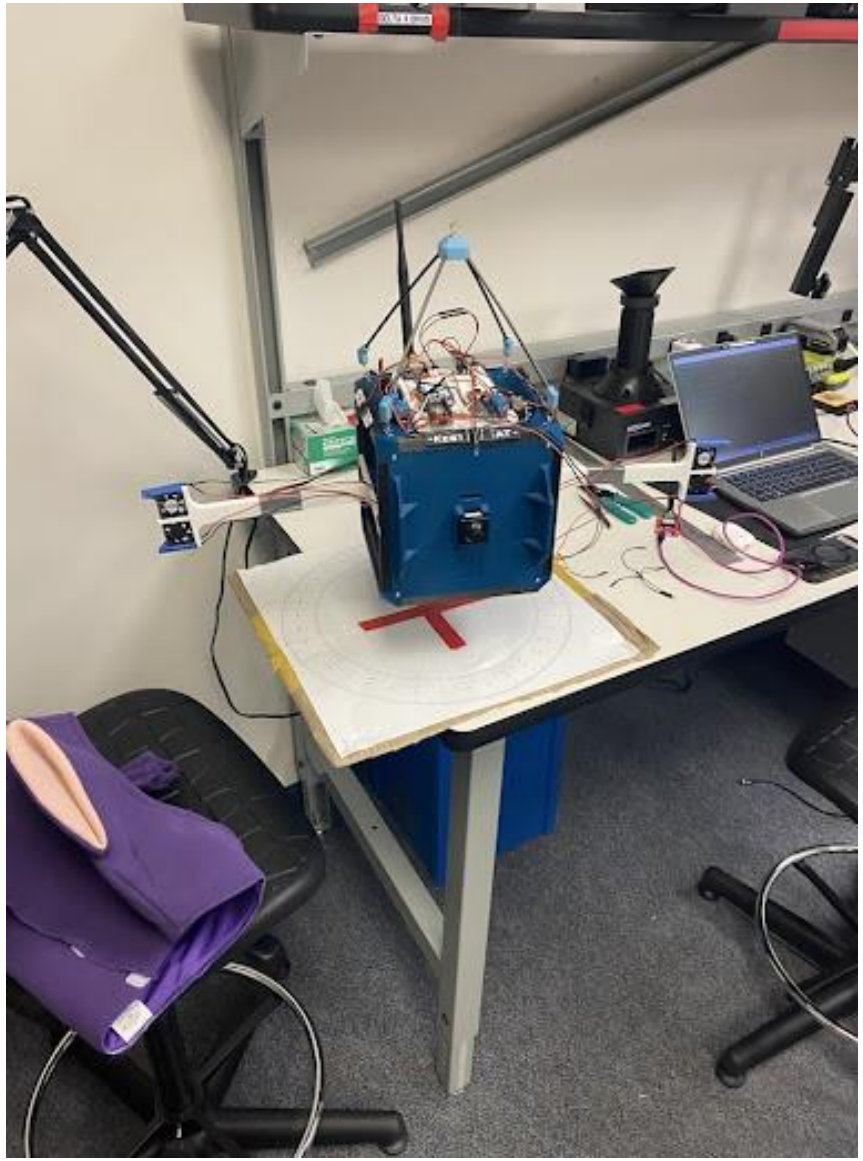
Astro-nots

Final Presentation

ASTRO 331 – Space Systems Engineering

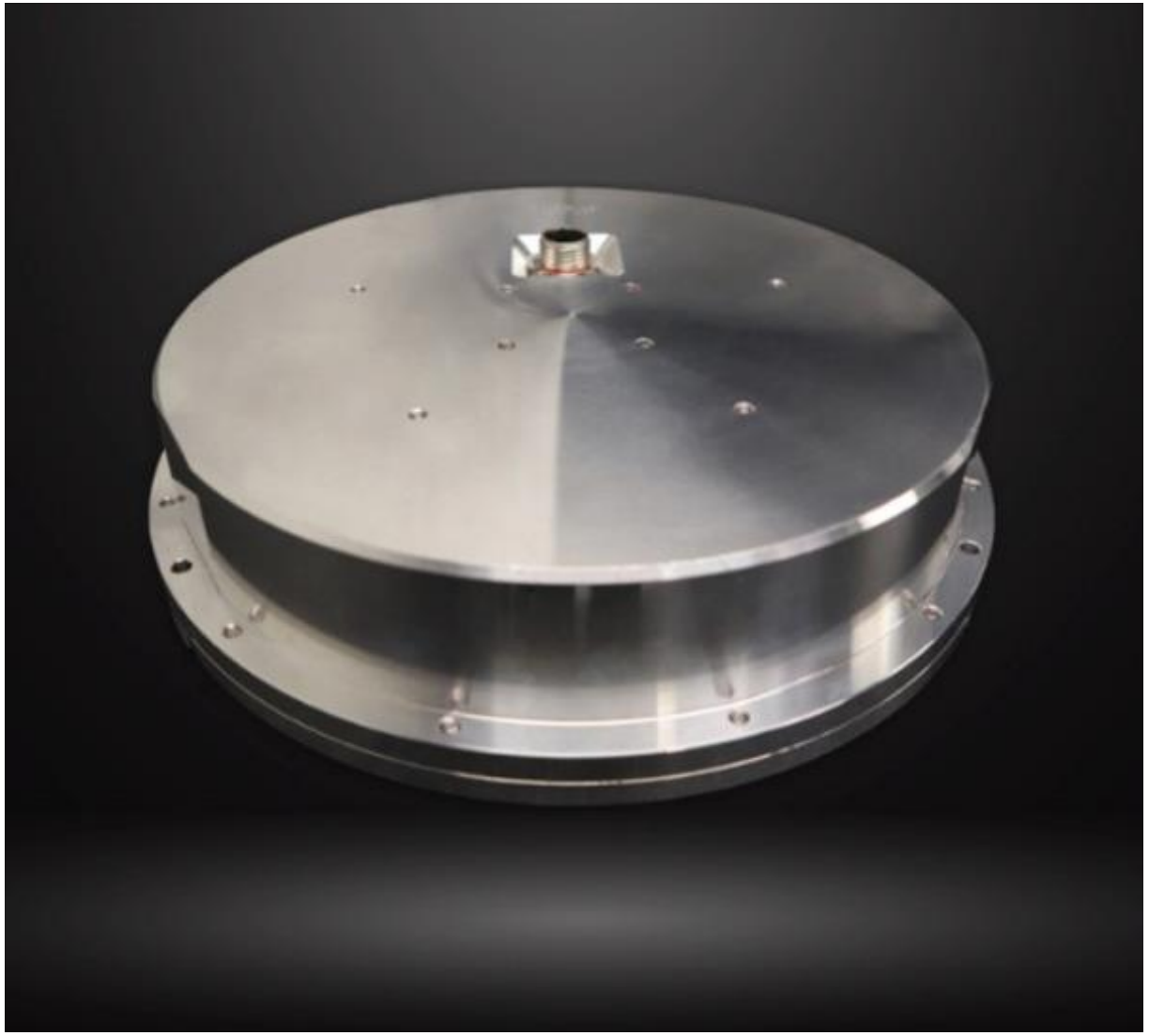
Overview

- Mission Objective
- Why this matters
- Requirements
- Data, Plots, & Analysis
- Engineering Outcomes



Mission Objective

- Imitates a mission that requires high power
- Solar array must be facing the sun at all times
- Reduces the mass of the satellite (less batteries)



Application – Why does it matter?

- Effective ADCS required for:
  - Spy satellites
  - GPS satellites
  - Orbital data centers
  - Communication



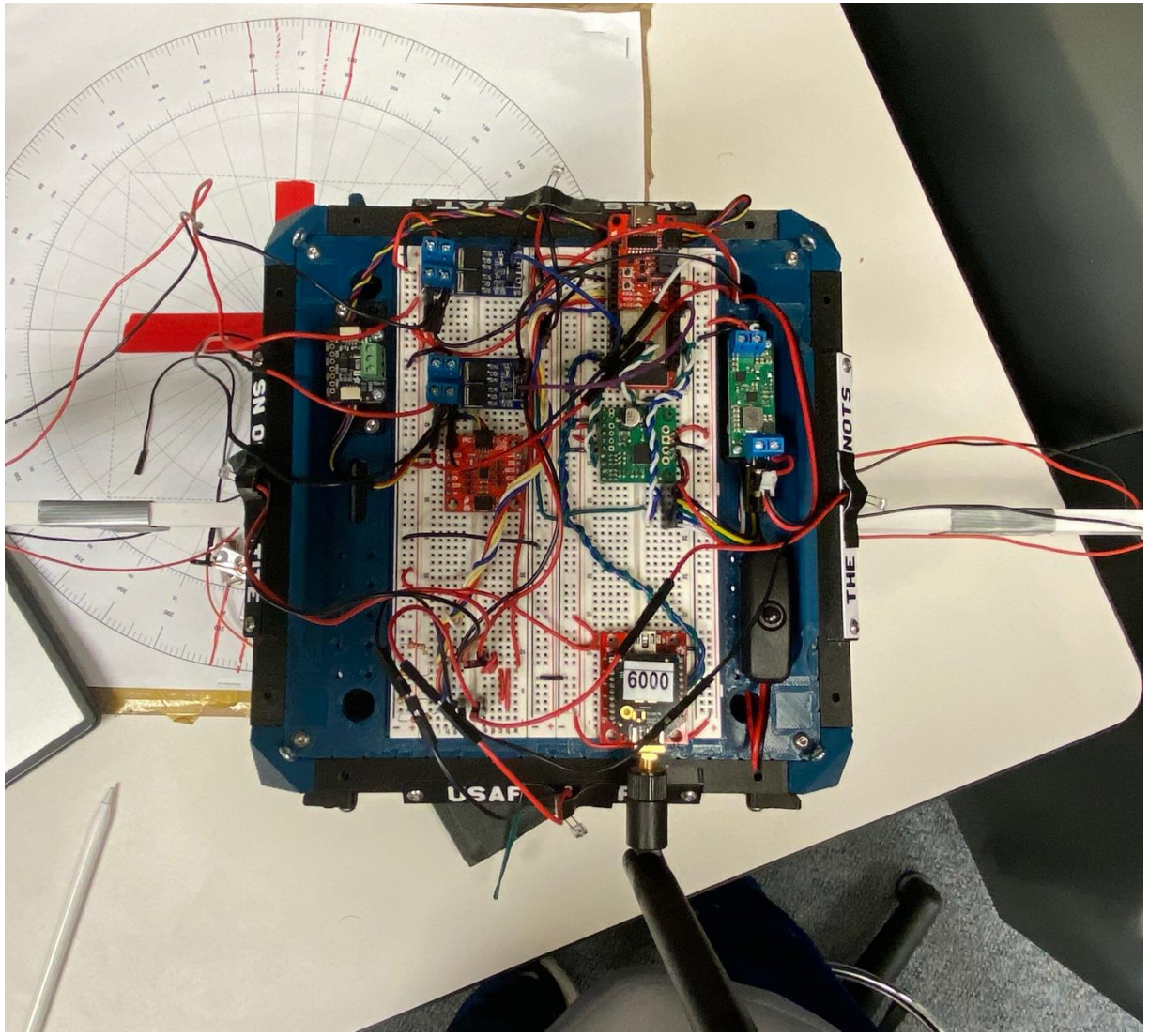
Requirements

Requirement	Verification Method	Status at Review
While in sunlight, KestrelSAT shall keep the solar array normal within ±10° of the Sun vector.	During testing, mount KestrelSAT and measure the distance between the solar array normal and the Sun vector. Monitor the distance to ensure that they are within 10° of each other.	Pass: KestralSAT maintains an accuracy within ±10° of the sun vector.
From an initial error of 180° away from the Sun vector, the KestrelSAT's solar array normal vector shall reach the Sun vector within 10 seconds, while maintaining the pointing accuracy requirement.	During testing, when there is an initial error of 180°, ensure KestrelSAT reaches the Sun vector within 10 seconds while maintaining the pointing accuracy requirement of 10°.	Pass: KestrelSAT's solar array normal vector reaches the sun vector in 3.2 seconds when starting from an initial error of 180° away from the sun vector.
The KestrelSAT shall maintain the pointing accuracy requirement while subject to external torques up to 2 Nm.	During testing, mount KestrelSAT and apply a 2 Nm disturbance torque using fans. Monitor the pointing accuracy to ensure that it is within the 10° pointing requirement.	Fail: 2Nm is an unrealistic force to both apply and counter to a satellite of this scale with our tools. We achieved this objective with 2mNm for roughly 60 seconds.
Stretch: While in Sun pointing mode, the KestrelSAT shall have a net positive battery charge rate.	Command KestrelSAT into Sun pointing mode and monitor the power system to ensure a net postitive battery charge rate.	Not achieved: we were not able to test this objective.
Stretch: The KestrelSAT should desaturate its reaction wheels while maintain the pointing accuracy requirement.	During testing, KestrelSAT successfully desaturates its reaction wheel. Monitor the pointing accuracy to ensure that it is within the 10° pointing requirement.	Met, able to achieve 8 momentum dumps at 70% fan power

Initial Approach

KestrelSAT Setup

- Communication: XBee3 antenna
- Sun sensors: Rapidly locate sun and provide data to reaction wheel
- Reaction Wheel: Quickly orient solar array towards sun
- Fans: Simulate disturbance torques



Code

- Core Architecture: Rate-limited PD controller utilizing gyro rate feedback (every 50 ms)
- Sensor Inputs:
  - Calculates pointing accuracy from the sun sensor (Proportional, pushing towards the sun)
  - Applies damping from the ICM-20948 gyro (Derivative, brakes the motion when it reaches the sun)
- Stable Acquisition:
  - Over-damped tuning guarantees the satellite locks onto the Sun without overshooting
  - Error of less than 1 degree is considered to be zero
- Active Bias Correction: An outer "lobe offset" loop continuously zeroes out steady pointing offsets, acting as a pseudo-integral term

Code – Sun Tracker

```
30  SunReading compute_sun(const float raw[4], const SunCalibration &cal) {
31    SunReading r;
32    r.magnitude = 0.0f;
33    for (int i = 0; i < 4; i++) {
34      float span = cal.bright[i] - cal.dark[i];
35      if (span < 1.0f) span = 1.0f;  // avoid divide by zero on a bad calibration
36      float n = (raw[i] - cal.dark[i]) / span;
37      if (n < 0.0f) n = 0.0f;        // darker than calibration = no light
38      if (n > 1.2f) n = 1.2f;        // brighter than calibration: cap it so one badly calibrated channel can't dominate
39      r.norm[i] = n;
40      r.magnitude += n;
41    }
42    r.sun_X = r.norm[SUN_PX] - r.norm[SUN_MX];
43    r.sun_Y = r.norm[SUN_PY] - r.norm[SUN_MY];
44    r.direction_deg = atan2f(r.sun_Y, r.sun_X) * DEG_PER_RAD;
45    if (r.direction_deg < 0) r.direction_deg += 360.0f;  // range 0–360 like Lab 6
46    return r;
47  }
```

← Takes multiple readings, averaging over the flicker of the lamp
← Normalizes the readings to have a value between 0 and 1
← Determines the XY coordinate and angle of the brightest spot

Code – PD Law

```
164  float tracker_step(float error_deg, float body_rate_dps, float dt_s,
165                     const TrackerGains &gains, const WheelLimits &limits,
166                     float I_wheel, float &w_cmd_rpm, bool &saturated) {
167    float error_rad = error_deg / DEG_PER_RAD;
168    float rate_rad_s = body_rate_dps / DEG_PER_RAD;
169
170    if (fabsf(error_deg) < gains.deadband_deg) error_rad = 0.0f;  // close enough: just damp
171
172    // Same as PD (tau = Kp*error - Kd*rate), written as "turn at a desired rate":
173    // desired rate = (Kp/Kd)*error, capped at max_rate so big slews are smooth and steady.
174    float tau;
175    if (gains.Kd > 0.0f) {
176      float rate_des = (gains.Kp / gains.Kd) * error_rad;
177      float max_rate = gains.max_rate_dps / DEG_PER_RAD;
178      if (rate_des >  max_rate) rate_des =  max_rate;
179      if (rate_des < -max_rate) rate_des = -max_rate;
180      tau = gains.Kd * (rate_des - rate_rad_s);
181    } else {
182      tau = gains.Kp * error_rad;
183    }
184
185    wheel_from_torque(tau, dt_s, limits, I_wheel, w_cmd_rpm, saturated);
186    return tau;
187  }
```

← 1 degree deadband

← Determines the desired torque to get to the target

← Turns the decided torque value into a wheel speed command

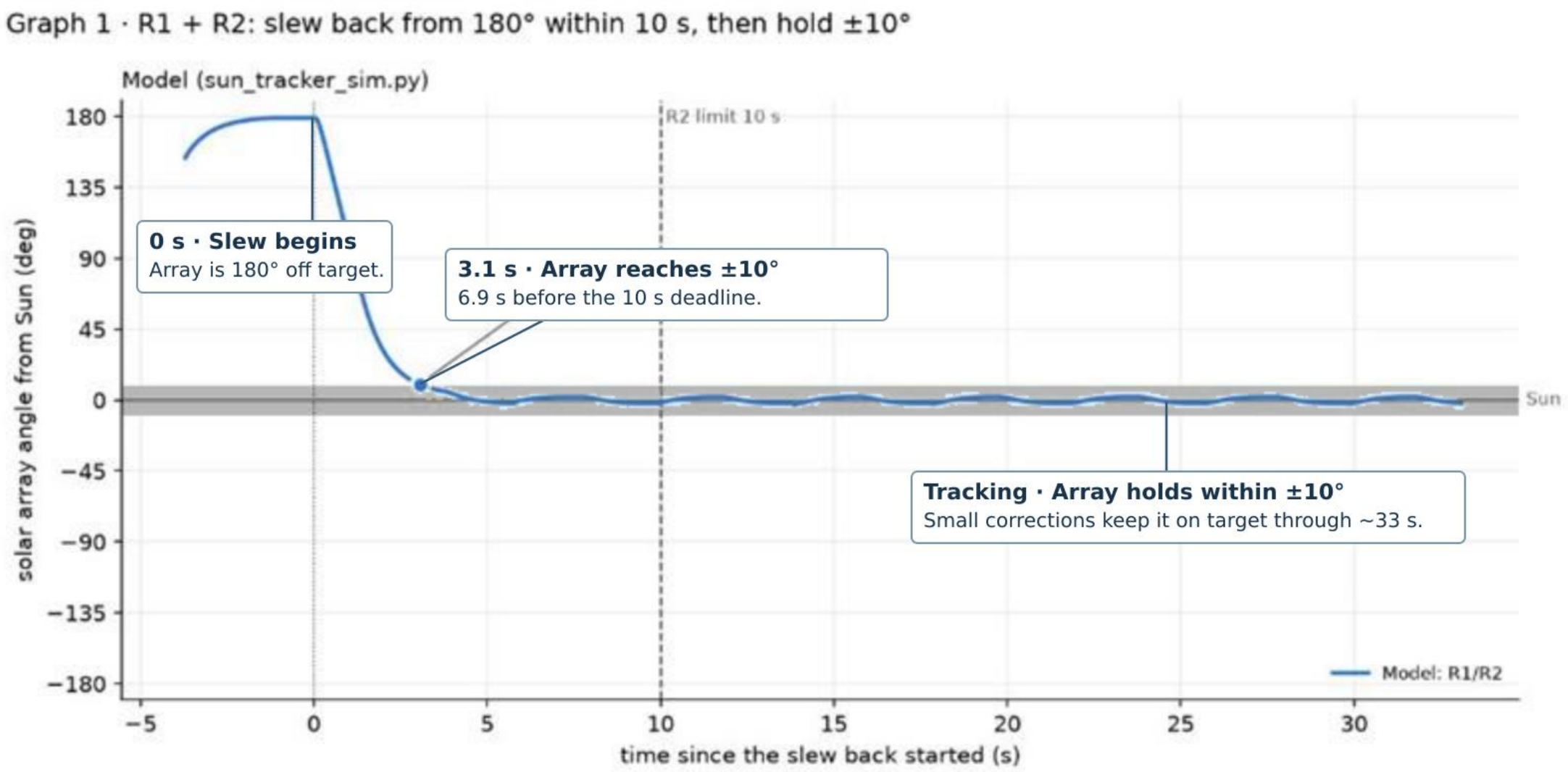
Code – PD Control to Wheel

```
140  static void wheel_from_torque(float tau, float dt_s, const WheelLimits &limits,
141                                float I_wheel, float &w_cmd_rpm, bool &saturated) {
142    float accel_rpm_s = (-tau / I_wheel) * RPM_PER_RAD_S;
143    if (accel_rpm_s >  limits.max_accel_rpm_s) accel_rpm_s =  limits.max_accel_rpm_s;
144    if (accel_rpm_s < -limits.max_accel_rpm_s) accel_rpm_s = -limits.max_accel_rpm_s;
145
146    w_cmd_rpm += accel_rpm_s * dt_s;
147
148    saturated = false;
149    if (w_cmd_rpm > limits.max_rpm) { w_cmd_rpm = limits.max_rpm; saturated = true; }
150    if (w_cmd_rpm < limits.min_rpm) { w_cmd_rpm = limits.min_rpm; saturated = true; }
151  }
```

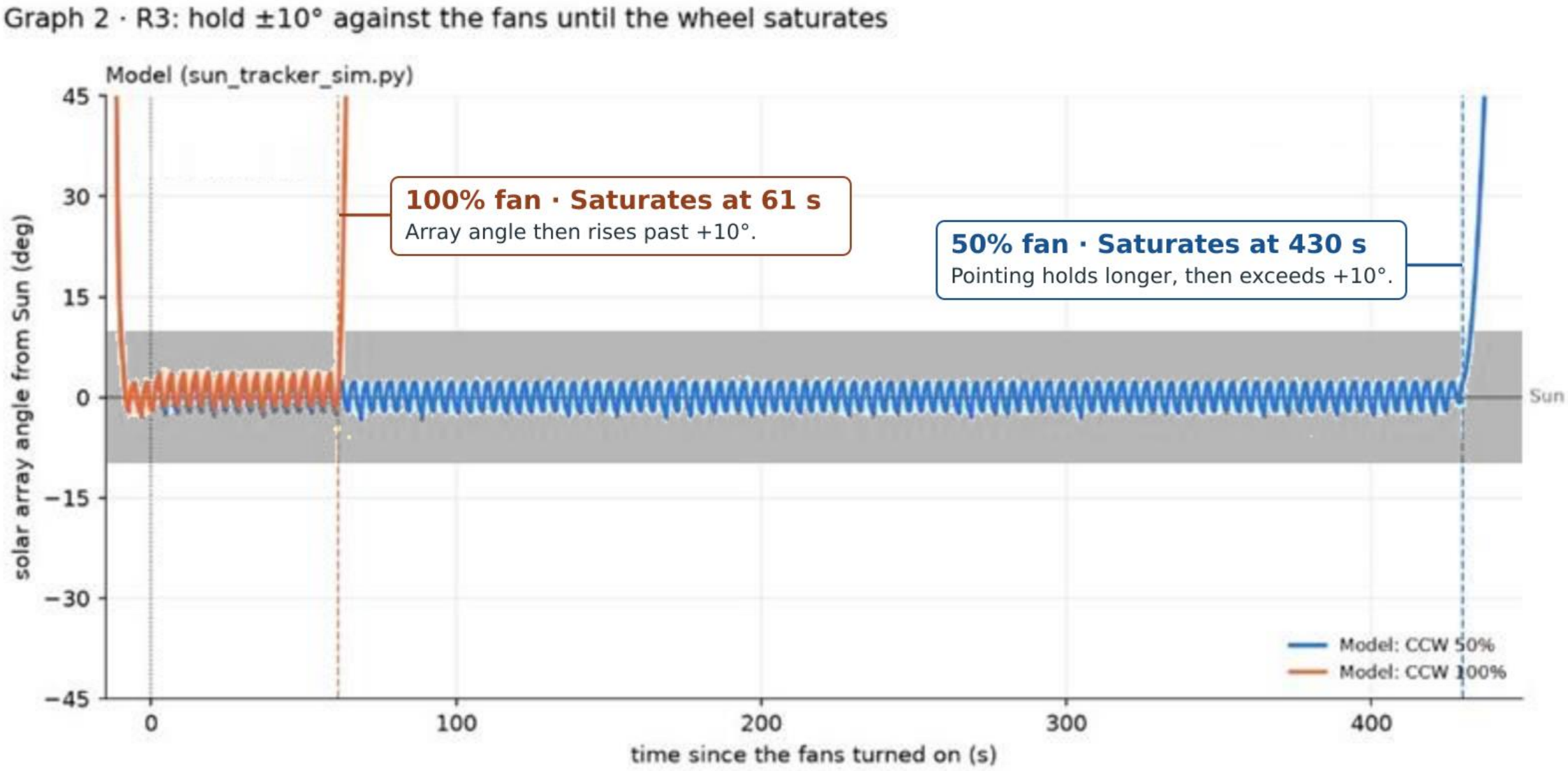
← Commands a turn of the reaction wheel in the opposite direction of the torque required

← Determines if the wheel is saturated

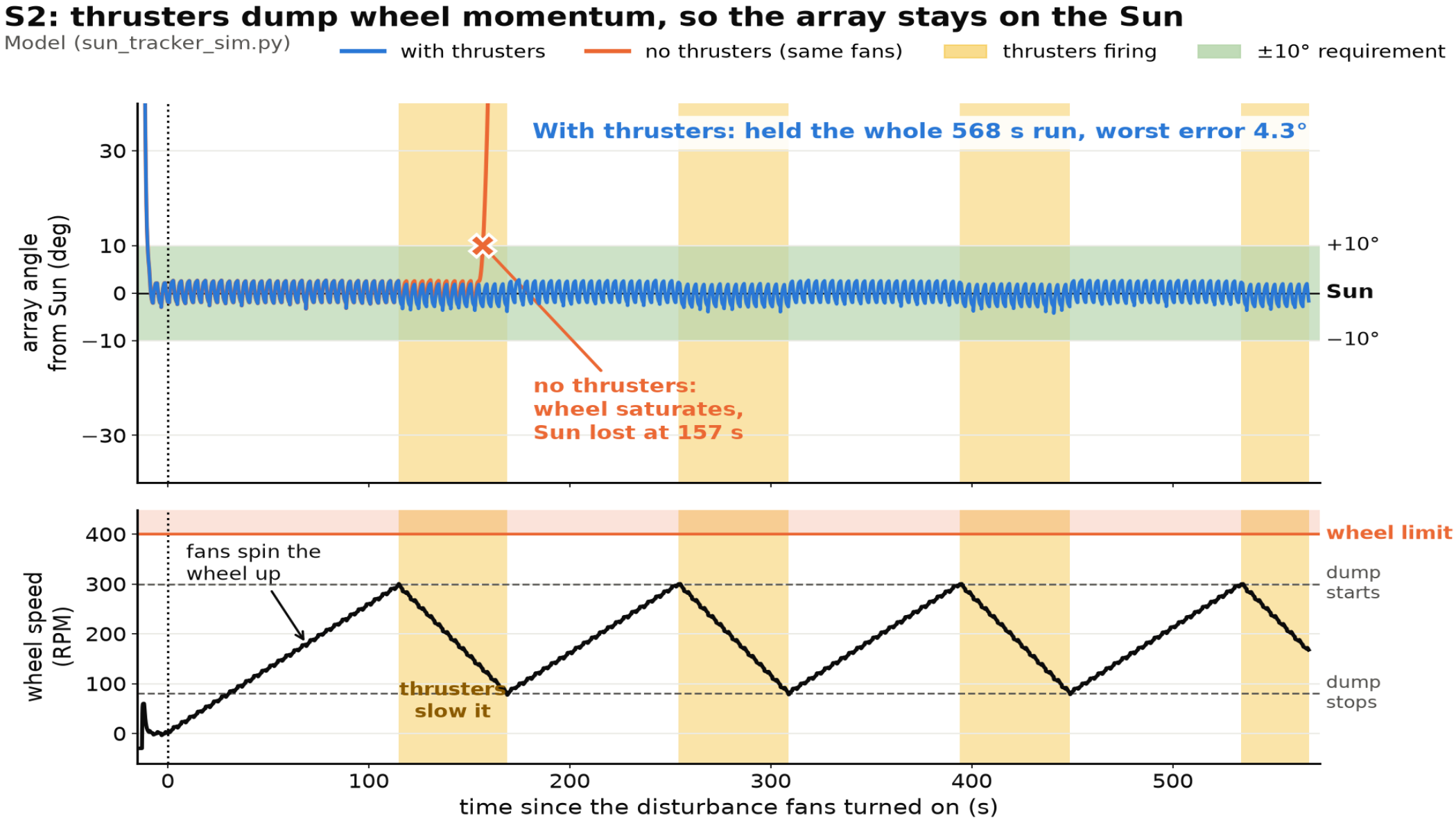
Theoretical Prediction – Req 1 and 2



Theoretical Prediction – Req 3



Theoretical Prediction – Stretch Req 2



Demonstration

Requirements 1 and 2

Testing Without Disturbance Torques

Requirements 1 and 2

Test: 1



Conditions: S/C starts with solar array 180 deg away from the sun

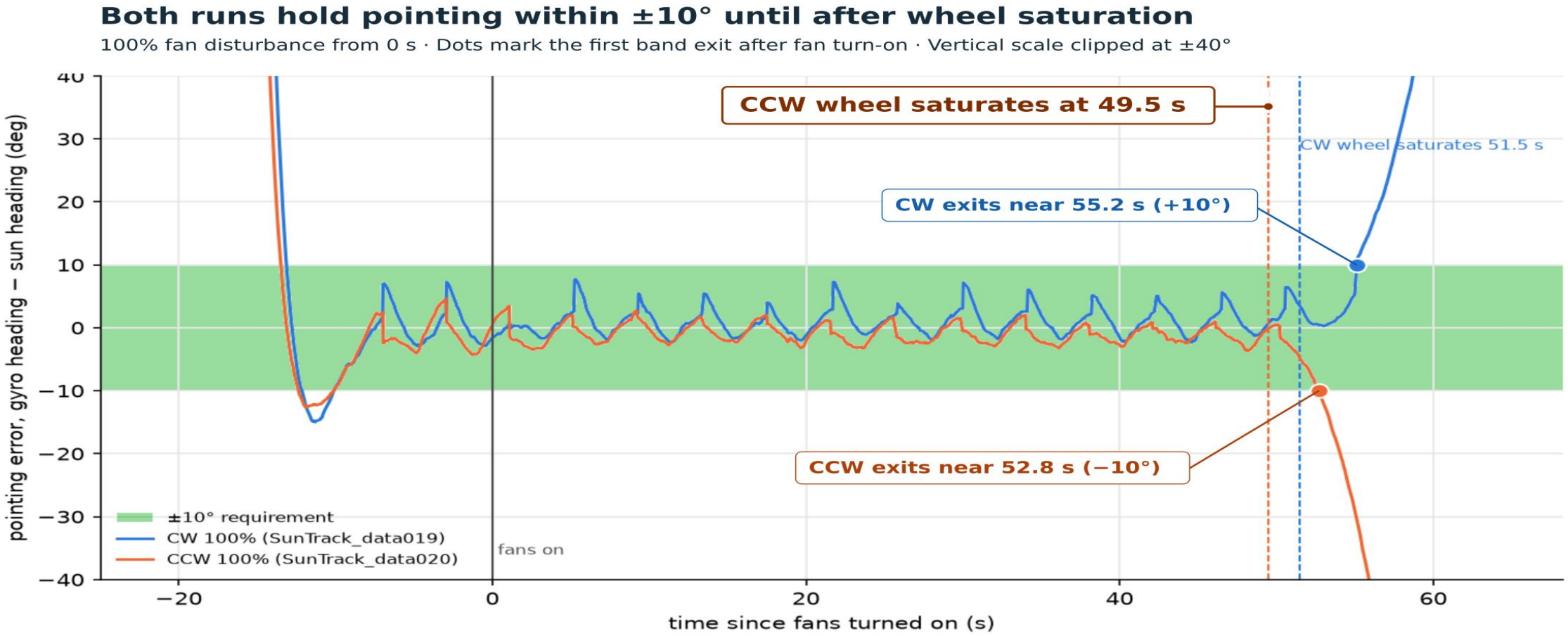
Results: S/C reaches sun at 3.2 sec

Improve: Build code for testing its ability to hold the 10 deg pointing accuracy while subject to disturbance torques

Testing with Disturbance Torques

Requirement 3

Test 2



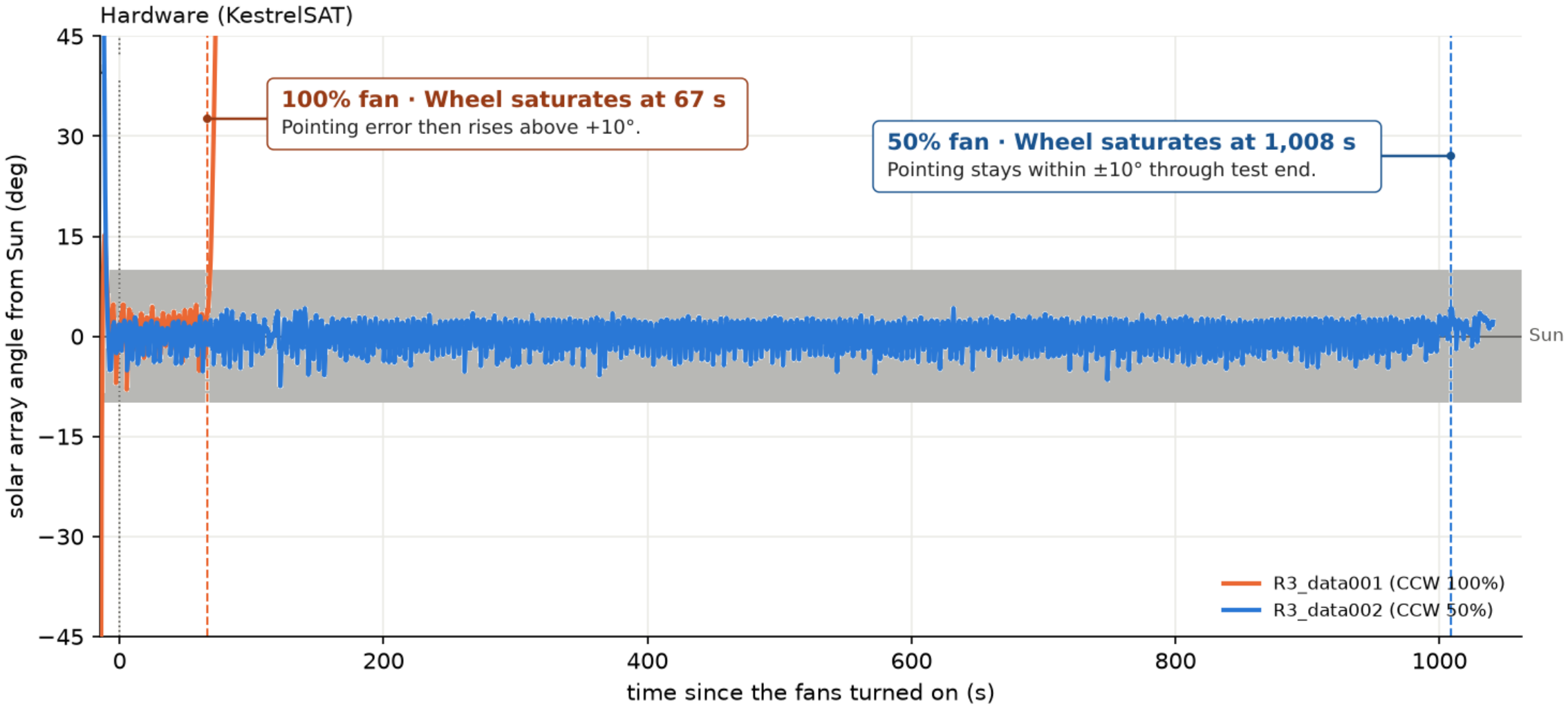
Conditions: Fans spin at 100% power after S/C achieves pointing accuracy

Results: S/C maintains accuracy for ~50s before reaction wheel saturation

Improve: Build code for desaturation using fans.

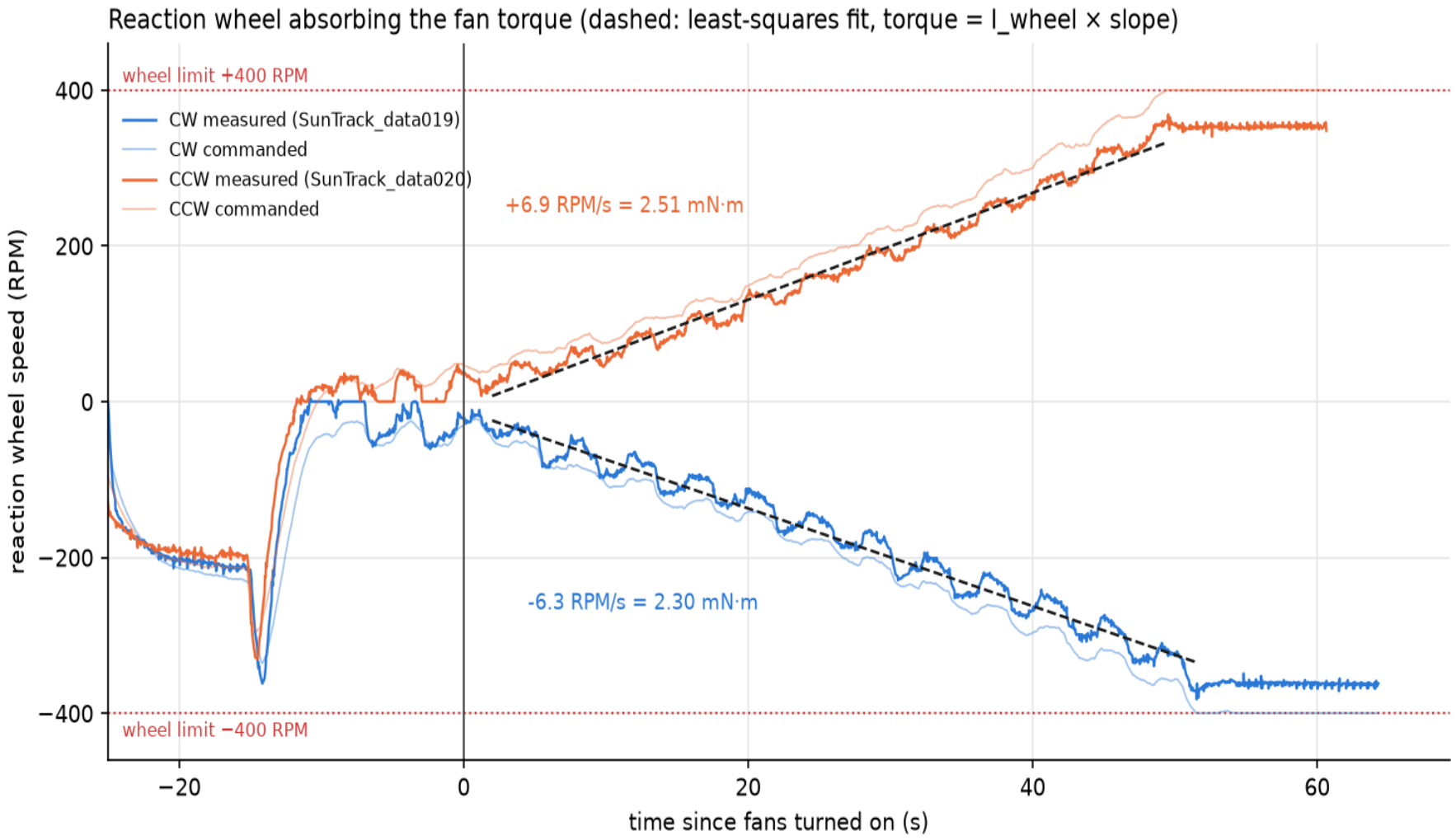
Test 3 – 50% vs 100% fan power

Graph 2 · R3: hold ±10° against the fans until the wheel saturates



Calculating the Disturbance Torque

Test 2



$\omega_w =$ Wheel Angular Velocity
$I_w =$ Wheel Moment of Inertia
$\tau_d =$ Fans Disturbance Torque

$$\tau_d = I_w \frac{d\omega_w}{dt}$$

	100%	50%
Test 3 (CCW only)	2.21 mN·m	0.18 mN·m
Test 2 (both directions)	2.30 / 2.51 mN·m	—
Model prediction	2.41 mN·m	0.34 mN·m

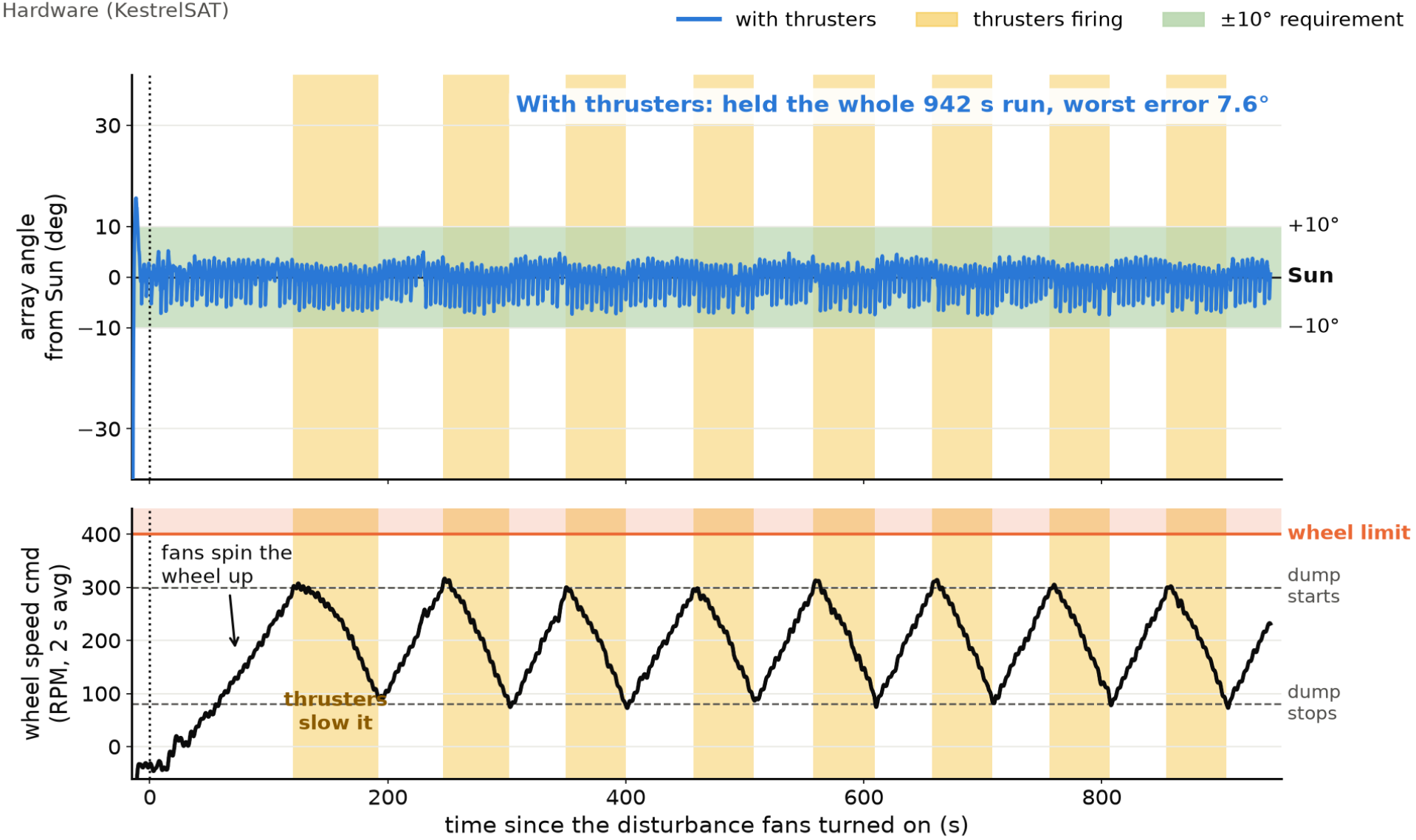
Testing Momentum Dumping

Stretch Requirement 2

Test 4

S2: thrusters dump wheel momentum, so the array stays on the Sun

Hardware (KestrelSAT)



Conditions:

- Fans spin at 70% power after S/C achieves pointing accuracy (~1.6mN m)
- After saturation, “thrusters” are powered to 100%

Results: S/C maintains accuracy for ~15 min and reaction wheel never saturates

Improve: Test with higher disturbance torques (better fans), Test longer periods of time

Failed Test



Conditions:
10kΩ resistance on all sensors
Testing ability to slew to sun.

Results:
S/C fails to commit to a direction.

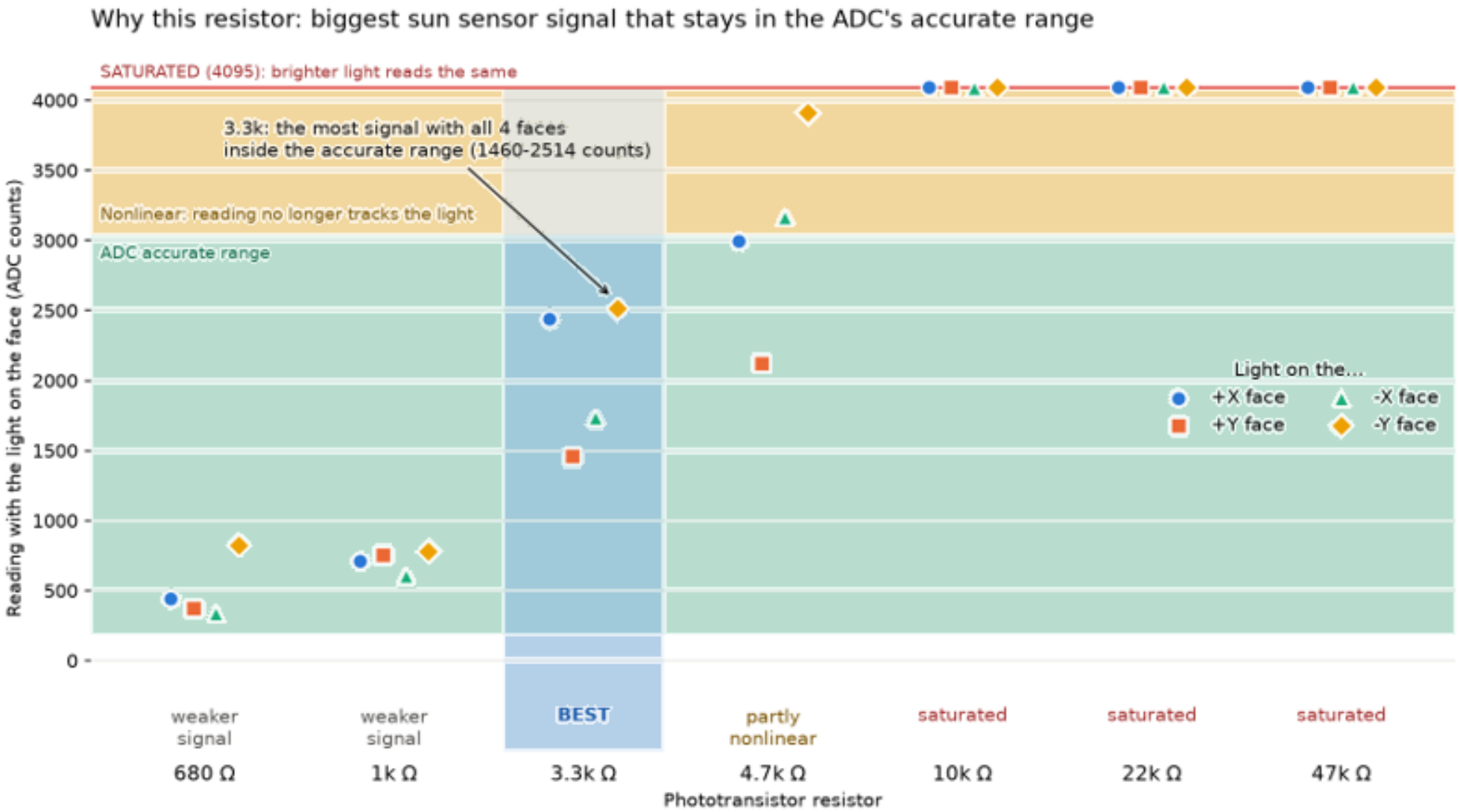
Improve:
We experimented changed out the resistors on the phototransistors.
Settling on 3.3kΩ.

Changed code to commit to one direction and begin tests with orientation loop to calibrate sensors.

Issues / Causes / Fixes

Previous Issues	Causes Investigated	How we fixed the issue
S/C was unable to achieve accuracy greater than 30°	- Disturbance torque from USB cable - Sun sensors oversaturated	- Ensure the satellite can be commanded by XBee radio - Test various resistors across the photo diodes - Fully clear the LOS of each sensor - Move each photo diode to each face, equidistant from the center
Reaction wheel does not spin up when under battery power	- Motor not receiving enough power (3.8V needs 4.5V) - Code errors	- Ensure fully charged battery levels before each test - Fixed wiring.
Satellite didn't commit to a direction. Getting stuck between 180° ±30°	- Code errors - Unstable sun tracking	- Fixed sun sensor - Increased sun sampling frequency and averaged the data points

Varying Resistance Levels on the Phototransistors



Engineering Outcomes

- Lessons Learned
  - Importance of neat wiring and organization early on
  - Testing many different solutions to find one that works
  - Testing Solutions one at a time
- Future Work
  - Using more power and better fans as thrusters to test higher torques
  - Monitoring the EPS to ensure net positive battery charge rate

Questions?